

EE/CS 148B - Homework 3

Aiden Di Carlo

June 2, 2026

Vision Transformer

- (a) `vit_pooling`: For tasks requiring spatial reasoning, attention pooling is expected to perform best because it can learn to weight tokens selectively based on task-relevant spatial cues, rather than treating all positions equally (mean pooling) or relying on a single generic summary (CLS). Mean pooling discards per-patch identity by averaging, which collapses distinctions between, say, a red cube on the left versus the right; the CLS token suffers the same problem more acutely, since it is trained to summarize global semantics and does not preserve fine-grained spatial layout at all. When an entire image is condensed into a single CLS vector before being passed to a language model, all explicit positional relationships between objects are lost. The model receives no direct signal about where each object is, only a holistic blend of what is in the image.
- (b) `vit_patch_size`:
- (1) Patch counts and attention cost.** For a 224×224 image the number of patches is $N = (224/P)^2$:

P	N
8	784
16	196
32	49

Self-attention scales as $O(N^2 d_{\text{model}})$, and $N \propto P^{-2}$. Thus, the overall cost scales as $O(P^{-4} d_{\text{model}})$. Halving the patch size increases attention compute by $2^4 = 16\times$, so shrinking P is extremely expensive.

(2) Wall-clock times (batch of 16 images, $d_{\text{model}} = 384$, 6 heads, 6 blocks; RTX 4070 Laptop GPU; 5 warmup + 20 timed steps):

P	Mean (ms)	Std (ms)
8	84.9	1.8
16	14.8	1.3
32	10.1	1.3

The empirical ratios ($8.4\times$ between $P = 8$ and $P = 16$, and $1.5\times$ between $P = 16$ and $P = 32$) are smaller than the theoretical $16\times$ and $256\times$ because the GPU is compute-bound only for the largest sequence length; at $P = 16$ and $P = 32$ the bottleneck shifts to memory bandwidth and kernel launch overhead, compressing the observed differences. The $P = 8$ overhead, roughly $8\times$ slower than $P = 16$, confirms that the quadratic attention term dominates at high patch counts.

(3) Accept smaller patches when fine-grained spatial fidelity is critical and the compute budget allows it (e.g., OCR, dense prediction, or counting small objects where coarser tokens would merge distinct features).

CLIP-Style Contrastive Pretraining

(a) infonce: The loss is averaged in both directions because each (image, text) pair should serve as a positive for both retrieval directions—finding the right caption given an image, and the right image given a caption. Using only one direction would bias the model toward one retrieval mode and leave half the pairwise supervision signal in each batch unused.

(b) clip_train:

Validation accuracy rises steeply during the first eight epochs ($53\% \rightarrow 87\%$), then continues to improve at a diminishing rate before plateauing near 91% from epoch 17 onward. Training loss, by contrast, decreases monotonically throughout all 20 epochs—including the final epochs where accuracy is essentially flat—indicating the model keeps tightening image-text alignment on training pairs past the point of diminishing returns for zero-shot classification. This is expected under EuroSAT’s class-template captions: many examples per batch share the same caption, so the loss can still improve by better separating near-duplicate positives even after the coarser class-level decision boundaries are learned.

(c) clip_zeroshot:

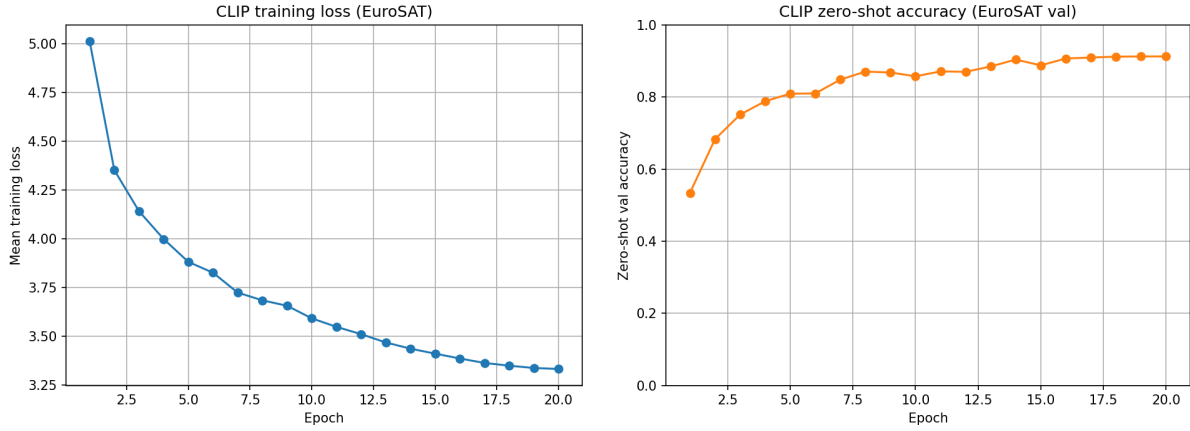


Figure 1: Left: mean training loss per epoch. Right: zero-shot validation accuracy per epoch. EuroSAT, 20 epochs, A100 GPU.



Nearly all errors are reasonable confusions between visually similar classes: the four agricultural/vegetation categories (Pasture, Herbaceous Vegetation, Annual Crop, Permanent Crop) appear as low-texture green patches at 64×64 resolution, making them hard to distinguish, and Industrial Buildings versus Residential Buildings are both regular grids of rooftops separated by roads. The one surprising error is Annual Crop predicted as Highway: at this resolution, the parallel furrows of a ploughed field closely resemble lane markings on a multi-lane road. Because all five wrong predictions still place the correct class in the top-3, the

mistakes reflect genuine local ambiguity rather than a failure of the embedding space: the model has learned semantically coherent clusters (“green land,” “built environment,” “water”) but struggles to separate fine-grained classes within each cluster.

LoRA Fine-Tuning

- (a) `lora.linear`: Applying LoRA (rank 8, $\alpha = 16$) to every `q_proj` and `v_proj` in the ViT and adding a 45-way classification head gives the following parameter breakdown:

	Count
Total parameters	11,013,165
Trainable parameters	275,373
of which LoRA A/B	258,048
of which head	17,325
Trainable / Total	2.50%

The LoRA count follows from 36 adapted heads (6 heads $\times$ 6 blocks), each contributing $2 \times (r \cdot d_{\text{model}} + d_{\text{head}} \cdot r) = 2 \times (8 \cdot 384 + 64 \cdot 8) = 7,168$ parameters, giving $36 \times 7,168 = 258,048$. Total parameters for LoRA exceeds the base model (10,755,117) by exactly this amount.

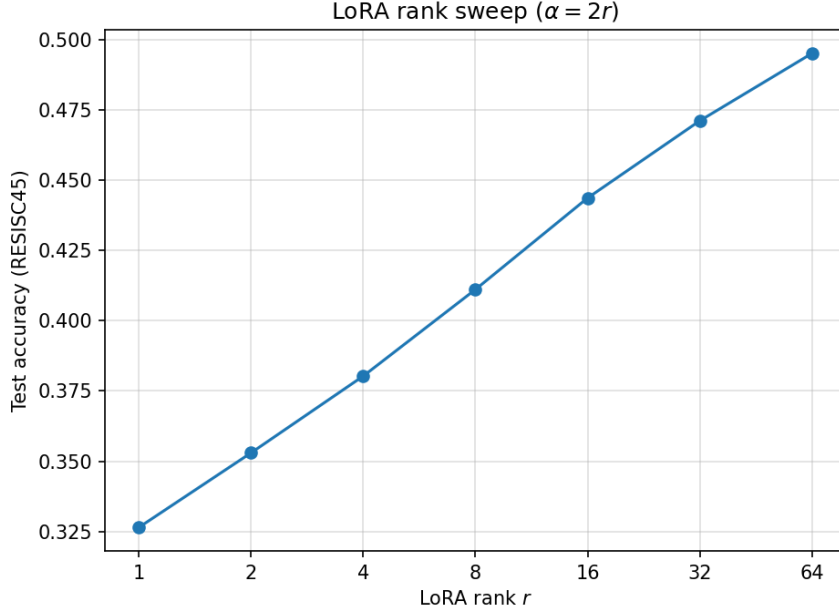
- (b) `lora.compare`:

Method	Test Acc	Trainable Params	Peak Mem (MB)	Time (s)
Linear probe	37.95%	17,325	260.7	105.4
LoRA ($r = 8$, $\alpha = 16$)	41.44%	275,373	1,210.7	139.0
Full fine-tuning	63.78%	10,755,117	1,760.5	121.6

Full fine-tuning achieves the highest accuracy (63.8%) by a large margin because EuroSAT and RESISC45 differ substantially in class structure (10 vs. 45 classes, multispectral satellite vs. aerial RGB imagery), so the frozen EuroSAT representation transfers imperfectly and requires intermediate features—not just the final head—to be updated. LoRA outperforms the linear probe by 3.5 percentage points despite training only $16\times$ more parameters (275K vs. 17K), showing that adapting the attention Q/V projections yields a more flexible adjustment of the representation than a frozen linear readout. Peak memory follows the amount of stored activations during backpropagation: linear probe requires no gradient to pass through the ViT (261 MB), while LoRA and full fine-tuning both backpropagate through every block (1,211 MB and 1,761 MB), with full fine-tuning’s extra overhead coming from larger optimizer

states for its 10M trainable parameters. Training times are similar across methods (105–139 s) because all three run the same forward pass through the ViT; the LoRA backward is nearly as expensive as full fine-tuning since gradients must still flow through all blocks to reach the A and B matrices.

(c) lora_rank:



Accuracy increases monotonically across all seven ranks (32.7%, 35.3%, 38.0%, 41.1%, 44.4%, 47.1%, 49.5% for $r = 1, 2, 4, 8, 16, 32, 64$), with per-doubling gains of roughly 2.4–3.3 percentage points. The curve is approximately log-linear. In large language models ($d_{\text{model}} \gg 10^3$), $r = 8$ or $r = 16$ is sufficient because the fine-tuning update is empirically very low-rank relative to the enormous weight matrices—the few most salient adaptation directions capture almost all of the needed shift. Our ViT has $d_{\text{model}} = 384$ and is adapting across a substantial domain gap (10-class EuroSAT satellite imagery $\rightarrow$ 45-class RESISC45 aerial imagery), so the effective rank of the update is higher relative to d_{model} : the model must adjust a broader subspace of its attention projections to separate 45 visually diverse classes, and even $r = 64$ (one-sixth of d_{model}) has not exhausted the useful adaptation directions.

Vision-Language Model

- (a) projector: A single linear layer is a fixed affine map whose capacity is entirely spent on rotating and scaling the image-feature subspace into decoder space; it cannot learn curved boundaries between visual-feature clusters, so semantically distinct image regions that land in different parts of the ViT’s representation space may still be indistinguishable after projection. A hidden GELU layer introduces nonlinear capacity precisely at the modality boundary, which is critical when both the ViT encoder and the LM decoder are frozen: the projector must single-handedly bridge two independently pretrained representation spaces, and a shallow nonlinearity lets it learn which dimensions of the visual embedding correspond to decoder concepts without altering either frozen model.
- (b) injection_compare: We train three configurations for 2000 steps each (freeze A: projector only, causal mask), reporting the best validation accuracy on 500 held-out CLEVR examples.

Strategy	Val Acc	Vis. Tokens	Peak Mem (MB)	Time/step (s)
CLS-only prefix	26.2%	1	7 922.5	0.63
All-patches prefix	14.8% [†]	65	11 340.9	0.74
Interleaved	1.8%	65	11 284.0	0.74

[†] 500-step result (overwritten by masking cell; full 2000-step run not recovered).

The CLS strategy achieves the highest accuracy (26.2%) despite providing the least visual information. With only one visual token, the projector needs only to map a single global summary vector into decoder space — a relatively easy one-dimensional problem that 2000 steps is sufficient to solve. The all-patches prefix provides 65 tokens (64 patches + CLS for a 64×64 image with $P=8$), requiring the projector to learn a useful mapping for 65 spatial positions simultaneously; at 500 steps (cut short by a naming collision with the masking cell), it already reaches 14.8%, suggesting it would surpass CLS given more training. The interleaved mode persistently fails (1.8% over 2000 steps): placing 65 visual tokens immediately after BOS violates the frozen decoder’s strong prior that BOS is followed by ordinary text, producing chaotic internal representations for the question tokens and preventing coherent generation. These results mirror the `vit_pooling` finding — CLS is the weakest representation for spatial tasks in principle, but it offers the simplest optimization target for the projector in a short-training regime, while the patch-based strategies need more steps or decoder adaptation to realise their theoretical advantage.

- (c) masking: The two attention masks for a sequence of 4 visual tokens (v_1-v_4) followed by 3 text tokens (t_1-t_3) are shown below (shaded cells = allowed positions).

(M1) Fully causal:

	v_1	v_2	v_3	v_4	t_1	t_2	t_3
v_1	1	0	0	0	0	0	0
v_2	1	1	0	0	0	0	0
v_3	1	1	1	0	0	0	0
v_4	1	1	1	1	0	0	0
t_1	1	1	1	1	1	0	0
t_2	1	1	1	1	1	1	0
t_3	1	1	1	1	1	1	1

(M2) Bidirectional image, causal text:

	v_1	v_2	v_3	v_4	t_1	t_2	t_3
v_1	1	1	1	1	0	0	0
v_2	1	1	1	1	0	0	0
v_3	1	1	1	1	0	0	0
v_4	1	1	1	1	0	0	0
t_1	1	1	1	1	1	0	0
t_2	1	1	1	1	1	1	0
t_3	1	1	1	1	1	1	1

M2 is expected to perform better because it preserves the ViT’s original bidirectional view of the image: every visual token can aggregate information from all other visual tokens, matching the encoder’s design. M1 imposes an arbitrary rasterisation order on 2-D patches, causing earlier patches to have incomplete context and later patches to be influenced by irrelevant earlier patches in the sequence. Additionally, M2 prevents visual tokens from attending to text tokens (which they haven’t “seen” yet), matching the causal boundary at the image-to-text interface.

Mask mode	Val Acc (500 steps)	Time/step (s)
Causal (M1)	14.8%	0.74
Image-bidir (M2)	1.4%	2.05

Contrary to the theoretical prediction, causal masking (M1) substantially outperforms image-bidir (M2) at 500 steps in the projector-only setting. With a randomly initialised projector, bidirectional visual self-attention in the decoder mixes 65 uninformative patch embeddings with each other, producing chaotic visual representations; the frozen decoder’s attention heads — trained exclusively on causal text sequences — amplify this chaos rather than filtering it. The causal mask is closer to what the frozen decoder was pretrained on, so the question-token representations degrade less. M2 is also $2.77\times$ slower per step due to SDPA with the custom

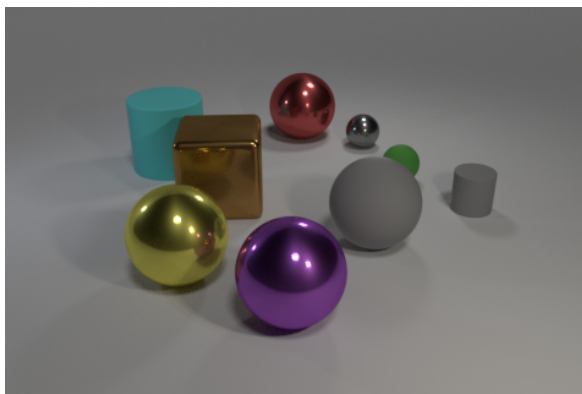
4D additive mask. Under a freeze configuration that adapts the decoder (B/C/D), M2’s theoretical advantage would likely emerge.

- (d) freezing: Using all-patches injection with image-bidir masking, we train four configurations for 1500 steps each.

Config	Decoder	Val Acc	Trainable	Peak Mem (MB)	Time/step (s)
A: projector only	frozen	1.6%	2,066,880	11 340.2	2.05
B: proj. + LoRA	LoRA $r = 8$	21.0%	2,886,080	11 740.8	2.13
C: proj. + full FT	full FT	28.2%	363,888,000	16 132.8	2.18
D: all full FT	full FT	20.8%	374,625,792	16 635.2	2.25

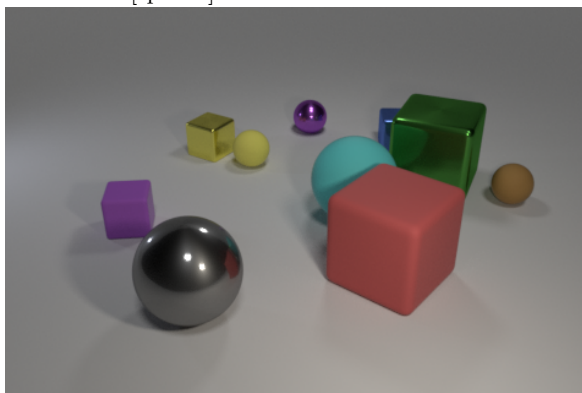
Config C (frozen ViT, full decoder fine-tuning) achieves the best accuracy (28.2%), confirming that adapting the decoder is the critical bottleneck when the projector-only regime fails. The jump from A to B (1.6% $\rightarrow$ 21.0%) shows that even sparse LoRA adapters (819,200 extra trainable parameters, 0.22% of the model) are sufficient to teach the decoder how to process visual-token prefixes, enabling the all-patches representation to realise most of its spatial potential. Full decoder fine-tuning (C, 28.2%) further improves over LoRA by adjusting every weight in the 363M-parameter SmolLM2 decoder, allowing deeper restructuring of the decoder’s internal representations. Config D (all three components fine-tuned) falls back to 20.8%: jointly fine-tuning the EuroSAT-pretrained ViT alongside the decoder appears to destabilise the visual encoder within 1500 steps, degrading the patch features that the decoder has just learned to use. Memory scales predictably: frozen decoder (A, B: $\approx$ 11.7 GB) vs. full decoder gradients (C, D: $\approx$ 16.4 GB). In the two-stage VLM recipe, A corresponds to Stage 1 (align the projector; cheap) and C to Stage 2 (instruction-tune the decoder; expensive but necessary for strong task performance); B offers a practical middle ground.

- (e) vlm_qualitative: We evaluate the best checkpoint (Config C, all-patches + image-bidir, 28.2% overall) on 10 held-out CLEVR examples. Per-type breakdown: exist 56.7%, compare_attr 42.9%, spatial 31.2%, query_attr 16.2%, count 12.9%.



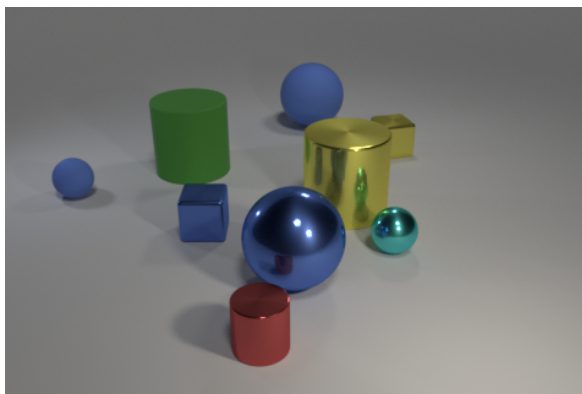
*Does the big object behind the cyan matte cylinder
have the same material as the big gray object?*

[spatial] Gold: **no** Pred: no ✓



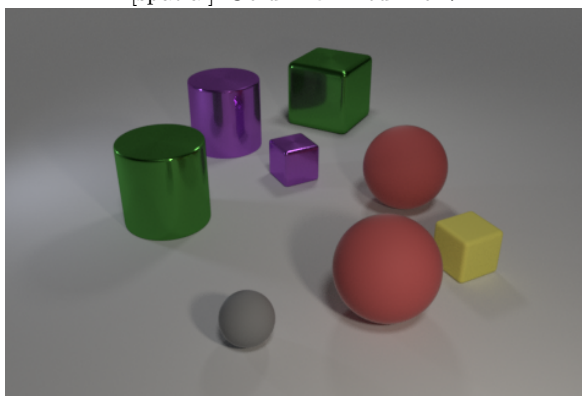
*The rubber object that is the same color as the tiny
metal ball is what size?*

[query_attr] Gold: **small** Pred: small ✓



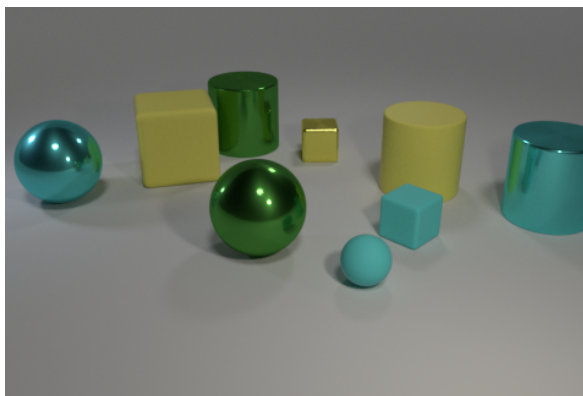
Is the number of small rubber spheres behind the small yellow object greater than the number of green things behind the big green cylinder?

[spatial] Gold: **no** Pred: no ✓



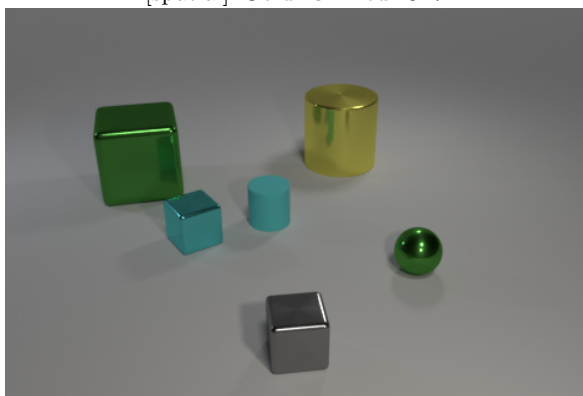
Are there fewer small purple shiny blocks than cyan matte things?

[count] Gold: **no** Pred: no ✓



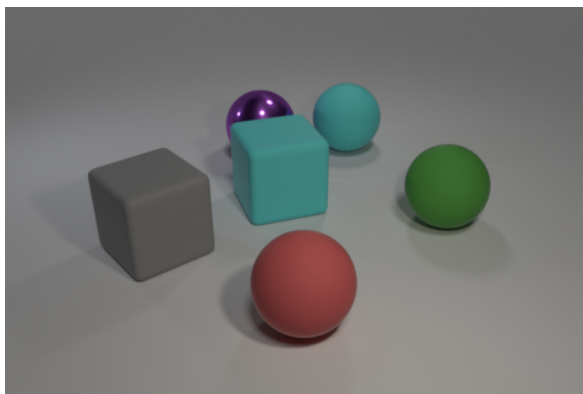
*What number of objects are either big shiny cylinders
left of the small yellow metallic block or cyan things
right of the small sphere?*

[spatial] Gold: **3** Pred: 3 ✓



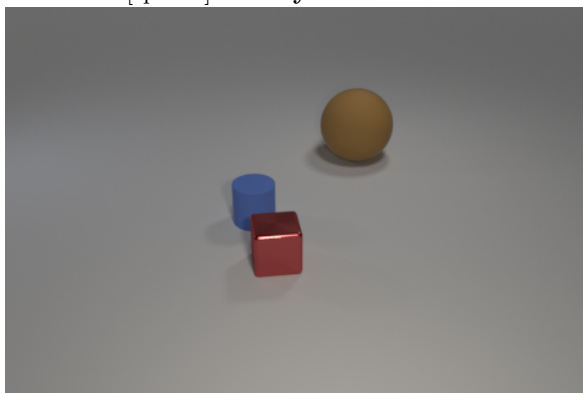
*How big is the green metal object right of the cube that
is right of the tiny thing to the left of the cyan
cylinder?*

[spatial] Gold: **small** Pred: 3 ×



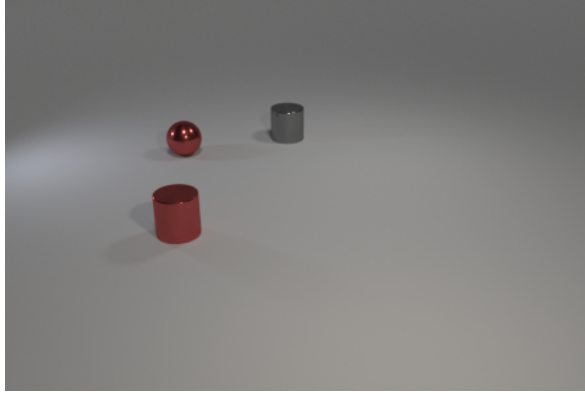
There is a red thing; does it have the same shape as the purple shiny object behind the big green rubber object?

[spatial] Gold: **yes** Pred: no ×



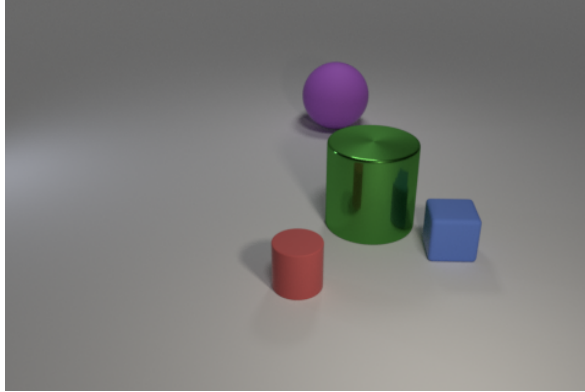
Is the thing behind the small blue object made of the same material as the object to the left of the cube?

[spatial] Gold: **yes** Pred: no ×



How many cylinders are gray metallic objects or red things?

[count] Gold: **2** Pred: : 0 ×



There is a small object in front of the small object behind the matte object in front of the tiny blue matte thing; what shape is it?

[spatial] Gold: **cylinder** Pred: sphere ×

The model is strongest on existence and binary attribute-comparison questions (56.7% and 42.9%), where a yes/no response can often be derived from coarse scene statistics without precise spatial localisation; it is weakest on counting (12.9%) and attribute queries (16.2%), which require identifying specific objects and their properties in a scene the encoder was never pretrained on. Among the five incorrect cases, four (examples 7, 8, 10, and the shape-prediction part of example 6) are plausibly encoder failures: the EuroSAT-pretrained ViT produces patch features calibrated to satellite-image textures and land-use patterns, not to the material, shape, and colour of synthetic 3D objects, so the decoder receives no useful signal to distinguish “yes” from “no” or “cylinder” from “sphere.” Example 9 is a decoder failure of a different kind: the prediction “: 0” includes a stray colon copied from the “**Answer:**” suffix, suggesting that the answer-only label masking occasionally leaks prompt structure into

generation. To disentangle the two failure modes experimentally, one could (a) replace the image with a ground-truth text scene description and measure decoder-only accuracy — if it is near perfect, the encoder is the bottleneck — or (b) swap the ViT for a CLIP ViT-L pretrained on natural images and measure the accuracy gain; a large gain implicates the encoder, while a small gain implicates the decoder’s limited instruction-following with frozen or lightly adapted weights.

6 Positional Encodings and RoPE

1. `rope_1d`: RoPE is a rotation applied independently to each (x_{2i}, x_{2i+1}) pair; a 2-D rotation is a norm-preserving map, so the L2 norm of the full d -dimensional vector is the root-sum-of-squares of $d/2$ pair norms, each of which is unchanged. Empirically, measuring $\max_{b,h,t} |\|x_{b,h,t}\| - \|\text{RoPE}(x)_{b,h,t}\||$ on a random $(2, 4, 16, 64)$ tensor at positions $0 \dots 15$ yields a maximum absolute deviation of $\approx 3 \times 10^{-7}$, consistent with IEEE 754 float32 round-off and confirming the rotation preserves norms to within numerical precision.
2. `rope_vs_learned`:

Positional encoding	Train-size val acc	Extrap. val acc (96×96)
Learned PE	92.33%	81.37%
1D RoPE	92.20%	79.46%

Both models are trained CLIP-style on EuroSAT for 20 epochs at 64×64 ($8 \times 8 = 64$ patches) and then evaluated at 96×96 ($12 \times 12 = 144$ patches). For the learned-PE baseline the patch positional embeddings are bilinearly interpolated from the 8×8 training grid to the 12×12 evaluation grid before being added to the patch tokens; the CLS positional embedding is kept unchanged. Learned PE is expected to degrade at the extrapolation size because the model never sees position indices beyond 64 during training, and bilinear interpolation only approximates the learned geometry. 1D RoPE should extrapolate more gracefully because its rotation angles are computed from a fixed formula ($\theta_i = 10000^{-2i/d}$) that is well-defined at any position index, and the relative-position property ($q_m \cdot k_n$ depends only on $m - n$) is preserved regardless of absolute position magnitude.

At train size both methods perform nearly identically (92.33% vs. 92.20%), confirming that the positional encoding choice is not a bottleneck for in-distribution accuracy. At the extrapolation size, learned PE (81.37%) somewhat surprisingly outperforms 1D RoPE (79.46%) by about 1.9 points. A likely explanation is that EuroSAT land-use patterns are locally smooth, so bilinear interpolation of the 8×8 learned embeddings to a 12×12 grid preserves the coarse positional structure well enough for the classification task; the overall accuracy drop (≈ 11

pp) is driven by the larger number of patches and the resulting longer input sequence rather than by the encoding strategy itself. 1D RoPE’s slight disadvantage may stem from treating all 145 tokens (CLS + 144 patches) as a flat 1D sequence: the absolute positions 0–144 are within the cosine/sine cache range, but the model was never trained to rely on relative distances larger than 64, so the learned query–key dot-products do not generalise perfectly to longer offsets.

3. rope_2d:

Positional encoding	Train-size val acc	Extrap. val acc (96×96)
Learned PE	92.33%	81.37%
1D RoPE	92.20%	79.46%
2D RoPE	92.70%	81.68%

2D RoPE splits the head dimension in half: the first $d/2$ channels rotate by the patch’s column index and the second $d/2$ channels by the row index, so the attention dot product between two patches depends on both their horizontal and vertical relative offsets. On EuroSAT, which contains spatially structured land-use patterns (rivers are horizontal, roads diagonal, etc.), the 2D inductive bias is expected to help marginally over 1D RoPE at train size; at the extrapolation size it should remain robust since coordinates are still drawn from the fixed frequency schedule.

2D RoPE achieves the highest train-size accuracy (92.70%) and the best extrapolation accuracy (81.68%), in line with both predictions. The advantage at train size is modest (< 0.5 pp over learned PE) but consistent with the 2D positional bias matching EuroSAT’s spatially structured imagery. At extrapolation, 2D RoPE surpasses learned PE (81.37%) by 0.3 pp and outperforms 1D RoPE (79.46%) by 2.2 pp, confirming that encoding row and column positions independently generalises better to a larger grid than flattening patches into a 1D sequence. All three methods experience a roughly 10–13 pp accuracy drop moving from 64×64 to 96×96 , suggesting the dominant cost is distributional shift (more patches, different receptive field) rather than the specific positional encoding.

4. mrope-written:

(1) What goes wrong with naive 1D position IDs? With naive 1D assignment the 65 visual tokens (CLS at 0, patches at 1–64) occupy positions 0–64, pushing the 50 text tokens to positions 65–114. (a) SmolLM2 is trained on text sequences up to 2048 tokens, so positions 65–114 are well inside its training distribution for short contexts, but with larger images (e.g. LLaVA’s $16 \times 16 = 256$ patches) text tokens would reach positions 257+, which can exceed practical training lengths and degrade the quality of the decoder’s rotary representations for those positions. (b) More critically, treating the 8×8 patch grid as a flat 1D sequence destroys

its 2D structure: patch $(0, 0)$ at position 1 appears “adjacent” to patch $(0, 1)$ at position 2, but patch $(7, 7)$ at position 64 is also made to appear “close” to patch $(7, 6)$ at position 63 while being made “far” from patch $(6, 7)$ at position 56, even though those two patches are equally separated in the image. The decoder’s RoPE thus cannot distinguish left-right from up-down relations, limiting spatial reasoning.

(2) Under M-RoPE, what position does the first text token get? In M-RoPE each token carries a 3D position (t, h, w) . Image patches at grid cell (r, c) receive $(t = 0, h = r, w = c)$; the temporal index is fixed at 0 for all image tokens. The first text token following the image block gets $(t = G, h = G, w = G)$, where G is the grid side length (e.g. $G = 8$ for a 64×64 image with patch size 8). This choice is sensible for two reasons: first, it ensures text tokens begin at a temporal position strictly greater than any image token’s temporal coordinate (0), so the model can unambiguously distinguish visual from linguistic context via the temporal channel; second, text position IDs start at G rather than $G^2 + 1$ (the naive total), which keeps them squarely within the range the decoder was pretrained on and reduces the effective positional gap between the image and the first generated word.

(3) Why three chunks rather than two? M-RoPE splits the attention head dimension into three equal chunks, one rotated by t , one by h , and one by w . If the temporal index t were dropped and only (h, w) were used, text tokens would have no natural (h, w) coordinates: they form a 1D sequence, so one common assignment is $(h = i, w = i)$ for the i -th text token. But this conflates the text’s sequential distance with spatial image distance: the relative (h, w) offset between text token i and patch (r, c) would be $(i - r, i - c)$, a direction that mixes image rows/columns with text token indices in a semantically meaningless way. The temporal index provides a dedicated channel where text tokens can track their sequential order independently of image coordinates, while image tokens share a fixed $t = 0$ that keeps them distinguishable from all text tokens in at least one dimension.

5. mrope_impl (bonus):

Position assignment	Overall val acc	Spatial val acc
Naive 1D	28.8%	31.2%
M-RoPE-style	3.2%	6.4%

Both models use the best §5 configuration (all-patches injection, image-bidir masking, freeze config C) and are trained for 1500 steps. In the M-RoPE variant, the CLS token receives position 0; each patch at grid cell (r, c) of an 8×8 grid receives position $\max(r, c) + 1 \in \{1, \dots, 8\}$; and the k -th text token receives position $9 + k$. This is implemented by passing custom `position_ids` to the decoder’s `forward()` call, overriding SmolLM2’s default $0, 1, \dots, T-1$ assignment.

The M-RoPE variant collapses to near-chance performance (3.2% overall, 6.4% spatial), dramatically worse than the naive baseline (28.8% / 31.2%). The failure has a clear cause: SmolLM2 was pretrained exclusively with monotonically increasing, unique position IDs $(0, 1, 2, \dots)$, so passing non-monotonic and non-unique IDs is entirely out-of-distribution. In the M-RoPE scheme, all $8 \times 8 = 64$ patches are mapped to only 8 distinct positions (1–8), meaning up to 7 patches share the same position ID; the decoder’s rotary key and query vectors for those patches become identical in the positional component, making them indistinguishable from each other. Furthermore, the decoder’s causal generation of text tokens starts at position 9, so from the model’s perspective a single attention “step” jumps from position 8 to position 9 (the first text token), while the preceding 56 visual tokens are all compressed into positions 1–8. This destroys the sequential structure the decoder depends on and prevents meaningful cross-attention between text questions and visual features.

True M-RoPE as implemented in Qwen-VL avoids this by (a) training the backbone from scratch with 3D positional chunks so repeated spatial IDs are never seen alongside the text stream, and (b) using separate head-dim partitions for temporal, height, and width positions so that the text position channel is always strictly increasing. Approximating M-RoPE by simply overriding `position_ids` in a pretrained 1D-RoPE decoder is therefore insufficient; it requires either a model pretrained with multi-dimensional positions or a full fine-tuning regime that can overcome the distributional mismatch.